

Sri Lanka Institute of Information Technology

Faculty of Computing

Dr.Sanvitha Kasthuriarachchi

IT2130 - Operating Systems and System Administration

Year 02 and Semester 02

Lecture 08

File System Handling in Operating System

Outline

- UNIX File Concept
- File Access Methods
- Disk and Directory Structure
- File Allocation Methods

File Concept

- A file system is a fundamental component of an operating system that manages how data is stored, organized, and retrieved from storage devices.
- It provides an abstraction layer between the user and the hardware, allowing users to interact with files without worrying about physical storage details.

File Concept

- A file is the smallest unit of logical storage in an operating system.
- It represents a collection of related information stored on secondary storage.
- Files can contain:
 - Text (documents)
 - Numbers (data files)
 - Binary data (images, videos)
 - Executable programs
- Each file is defined by its creator and has a specific purpose. The operating system treats all files as a sequence of bytes or records.

File Attributes

- Each file has associated metadata known as attributes. These help the operating system manage and control files effectively.
- Key attributes include:
 - **Name:** Human-readable identifier
 - **Identifier:** Unique internal ID
 - **Location:** Where the file is stored
 - **Size:** Current file size
 - **Protection:** Access permissions
 - **Time & Date:** Creation and modification timestamps
- These attributes are essential for security, tracking, and file organization.

Directory Structure

- A directory is a special type of file that stores information about other files. It acts like an index or table of contents.
- Functions of directories:
 - Organize files logically
 - Store file metadata
 - Enable efficient searching
- Directories make file management easier and more structured.

File Operations

- The operating system provides several operations to manipulate files:
 - **Create:** Generate a new file
 - **Open:** Prepare file for access
 - **Read:** Retrieve data
 - **Write:** Modify or add data
 - **Seek:** Move within the file
 - **Close:** Finish operations
 - **Delete:** Remove file
- These operations are implemented using system calls.

Open Files

- Several pieces of data are needed to manage open files:
 - **Open-file table**: tracks open files
 - File pointer: pointer to last read/write location, per process that has the file open
 - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
 - Disk location of the file: cache of data access information
 - Access rights: per-process access mode information

Open Files

- When a file is opened, the OS maintains information in an **open-file table**.
- This includes:
 - File pointer (current position)
 - Access rights
 - Number of processes using the file
- This mechanism improves performance by avoiding repeated disk access and supports multi-user environments.

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

File Access Methods

- File access methods define how data inside a file is accessed.
- Main types:
 - **Sequential Access** – data is accessed in order
 - **Direct Access** – data can be accessed randomly
- The choice depends on application requirements.

Sequential Access

- In sequential access, data is read or written in order, from beginning to end.
- The characteristics are:
 - Simple to implement
 - Efficient for linear data processing
 - Not suitable for random access
- Example: Reading a log file line by line.

Direct Access

- Direct access allows reading or writing at any location in the file.
- The characteristics are:
 - Fast and flexible
 - Requires addressing mechanism
 - Used in databases and large systems
- Example:
Accessing record number 100 directly.

Operations Performed on Directory

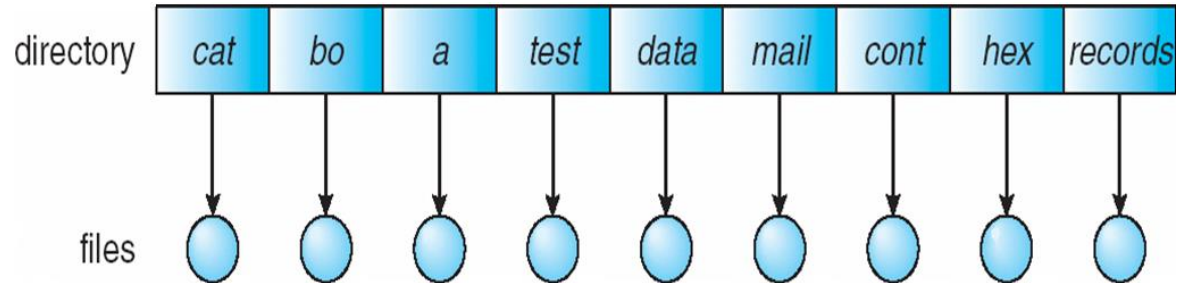
- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

Types of Directories

- Different directory structures are used to organize files:
 - Single-level directory
 - Two-level directory
 - Tree-structured directory
 - Graph-based directory

Single-Level Directory

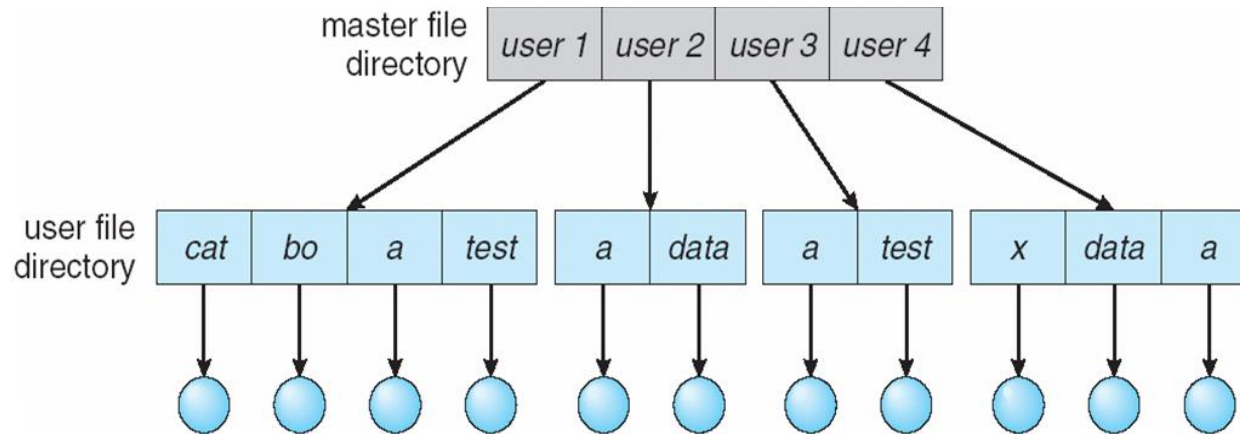
- A single directory for all users



- Naming problem
- Grouping problem

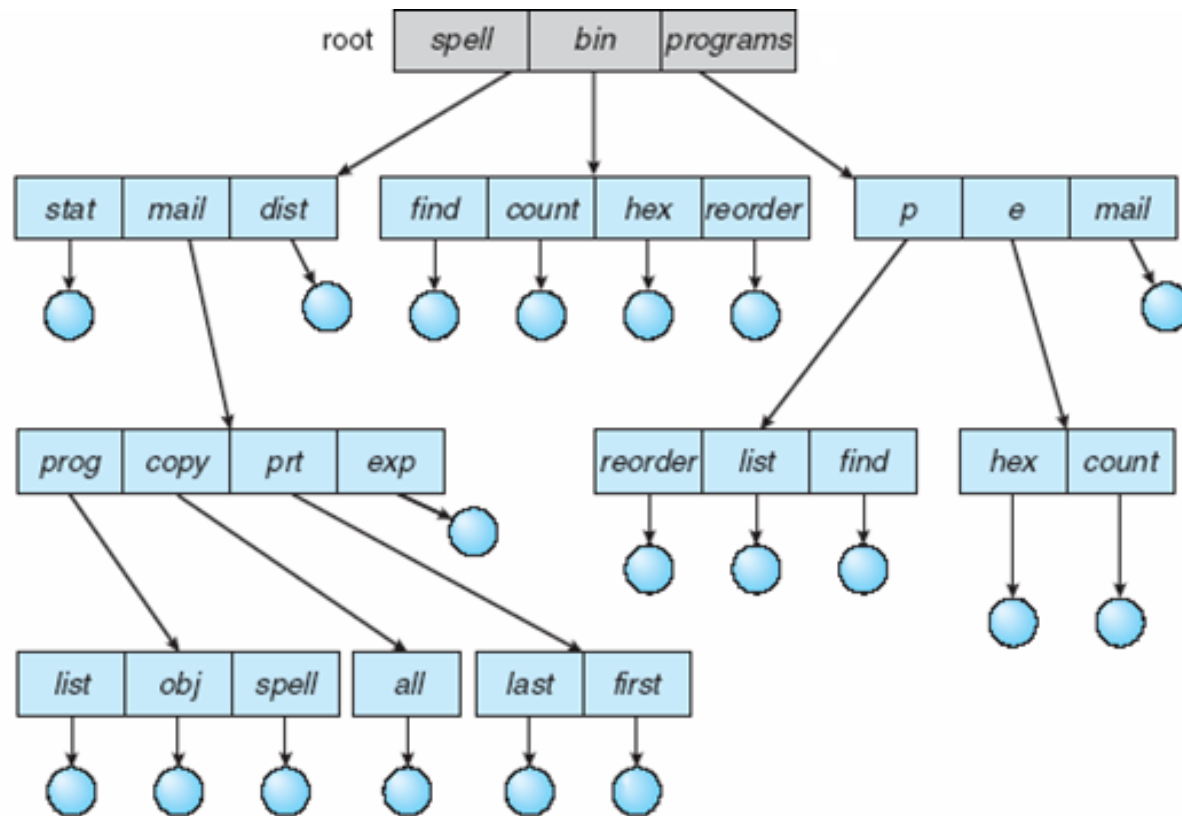
Two-Level Directory

- Separate directory for each user

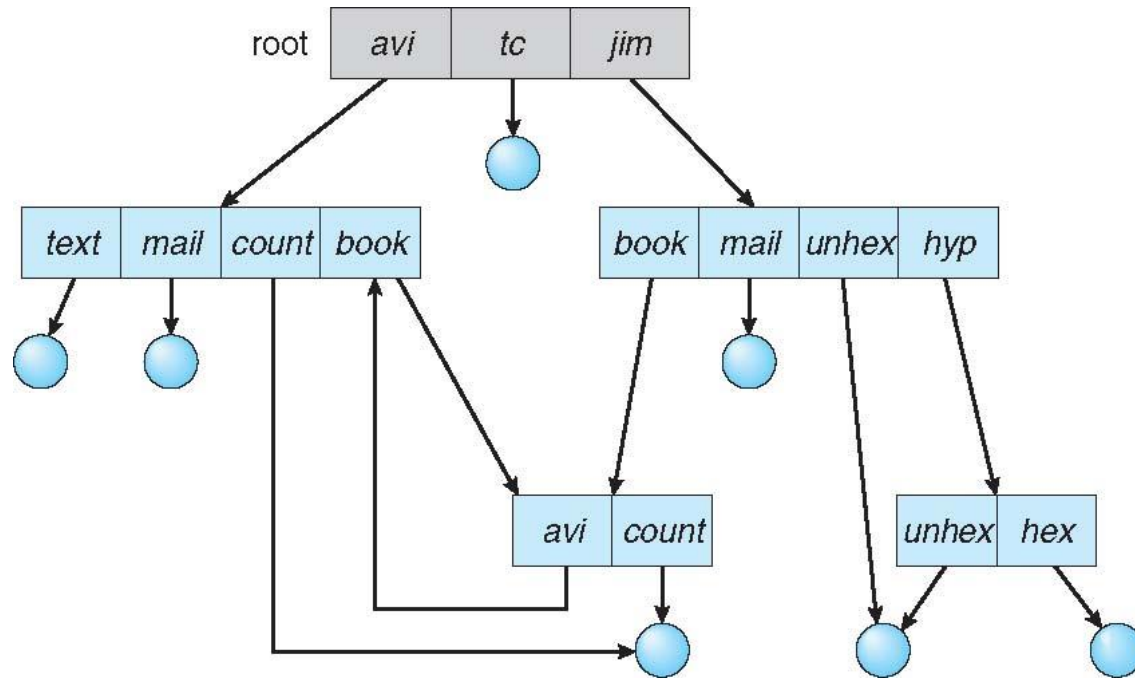


- Path name
- Can have the same file name for different user
- Efficient searching
- No grouping capability

Tree-Structured Directories



General Graph Directory



File Protection

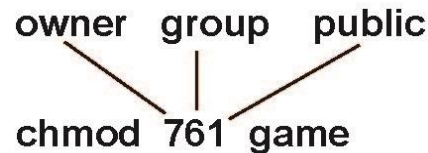
- File protection ensures that only authorized users can access files.
- Types of access:
 - Read
 - Write
 - Execute
- Protection mechanisms are critical in multi-user systems.

Access Lists and Groups in Unix

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

a) owner access	7	⇒	RWX 1 1 1 RWX
b) group access	6	⇒	1 1 0 RWX
c) public access	1	⇒	0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a file (say *game*) or subdirectory, define an appropriate access.



- Attach a group to a file

chgrp **G** **game**

A Sample UNIX Directory Listing

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

File-System Structure

- A file system is organized in layers:
 - Application layer
 - Logical file system
 - Physical file system
 - Device drivers
- This layered design improves modularity and efficiency.

File Control Block (FCB)

- OS maintains **FCB** per file, which contains many details about the file
 - Typically, inode number, permissions, size, dates
 - Example

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

File Allocation Method

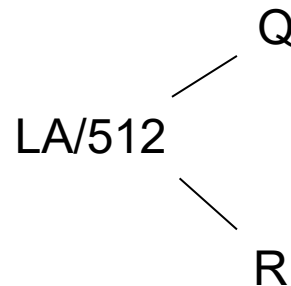
- An allocation method refers to how disk blocks are allocated for files:
 - Contiguous
 - Linked
 - File Allocation Table (FAT)

Contiguous Allocation Method

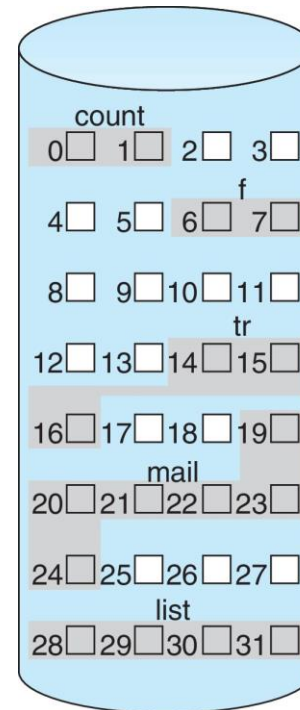
- An allocation method refers to how disk blocks are allocated for files:
- Each file occupies set of contiguous blocks
 - Best performance in most cases
 - Simple – only starting location (block #) and length (number of blocks) are required
 - Problems include:
 - ▶ Finding space on the disk for a file,
 - ▶ Knowing file size,
 - ▶ External fragmentation, need for **compaction off-line (downtime)** or **on-line**

Contiguous Allocation (Cont.)

- Mapping from logical to physical
(block size = 512 bytes)



- Block to be accessed = starting address + Q
- Displacement into block = R



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Contiguous Allocation

- In contiguous allocation, as Blocks are stored sequentially and Address is calculated directly 1 I/O operation is required to allocate a single data block.
- A file is stored using contiguous allocation. How many I/O operations are required to access the 8th block? Direct access → 1 I/O
- A file contains 12 blocks. How many I/O operations are required to read the entire file? Read 12 blocks → 12 I/O

Extent-Based Systems

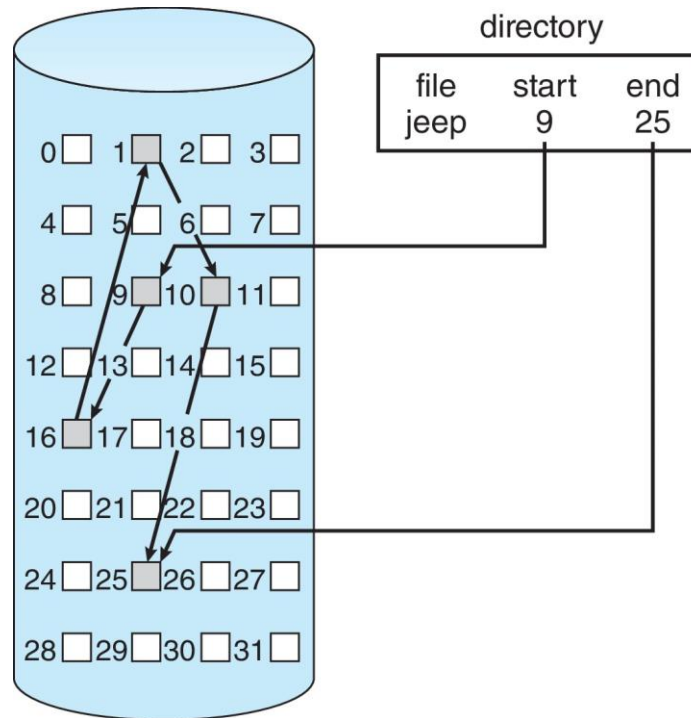
- Many newer file systems (i.e., Veritas File System) use a modified contiguous allocation scheme
- Extent-based file systems allocate disk blocks in extents
- An **extent** is a contiguous block of disks
 - Extents are allocated for file allocation
 - A file consists of one or more extents

Linked Allocation

- Each file is a linked list of blocks
- File ends at nil pointer
- No external fragmentation
- Each block contains pointer to next block
- No compaction, external fragmentation
- Free space management system called when new block needed
- Improve efficiency by clustering blocks into groups but increases internal fragmentation
- Reliability can be a problem
- Locating a block can take many I/Os and disk seeks

Linked Allocation Example

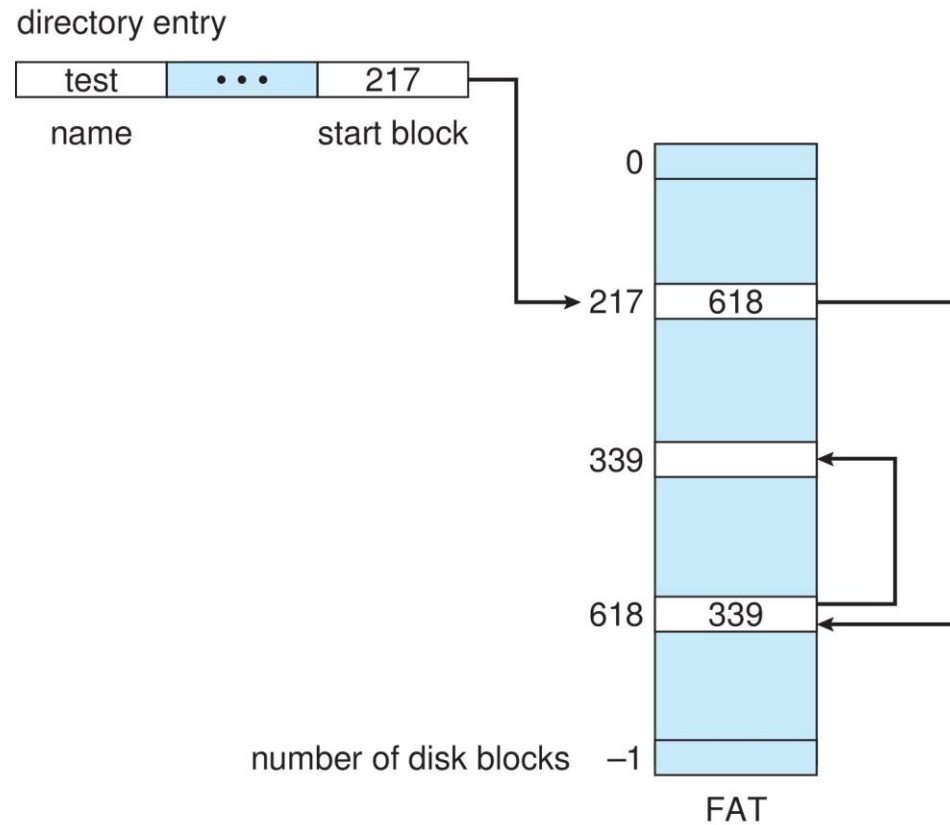
- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk
- Scheme



Linked Allocation

- A file has 10 blocks stored using linked allocation. How many I/O operations are needed to access the 6th block? Must traverse first 6 blocks → 6 I/O
- How many I/O operations are required to read the entire file of 8 blocks? → 8 I/O

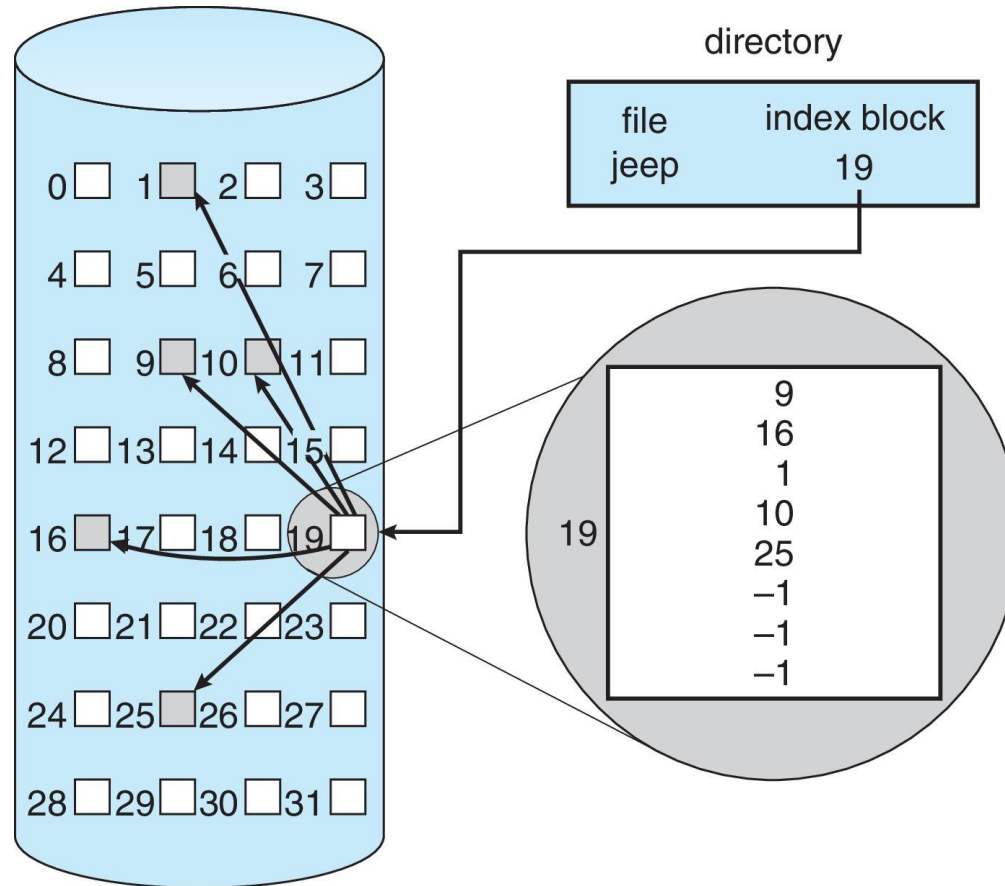
File-Allocation Table



Indexed Allocation Method

- Each file has an index block containing pointers.
- Advantages:
 - Supports direct access
 - Efficient for large files
- Disadvantages:
 - Extra storage overhead

Example of Indexed Allocation



Index Allocation

- Step 1 → Read index block
- Step 2 → Read data block

- A file uses indexed allocation. How many I/O operations are required to access the 4th block?
 - Read Index block → 1 I/O
 - Read Data block → 1 I/O

2 I/Os
- If the index block is already in memory, how many I/O operations are needed?
Only data block access → **1 I/O**

- A file has 10 blocks. How many I/O operations are required to read the entire file?
 - Read Index block → 1 I/O
 - Read Data blocks → 10 I/Os **$10 + 1 = 11$ I/Os**

Example

- A file contains 10 blocks. Calculate I/O operations required to access the 7th block in:
 - Contiguous allocation
 - Linked allocation
 - Indexed allocation

Answer

- Contiguous: 1 I/O
- Linked: 7 I/Os
- Indexed: 2 I/Os

Modern File Systems

- Modern File Systems are highly reliable and support for large storages.
- Examples include:
 - NTFS (Windows)
 - ext4 (Linux)
 - APFS (macOS)

File System Performance

- Performance depends on:
 - Disk speed
 - Allocation method
 - Caching mechanisms
- Modern systems use buffer caches to improve efficiency.

End of Lecture 8



Weekly plan

- **Practical** : File Management in UNIX
- **Workshop**:
 - Summary – File management
 - Discussion of Tutorial 8

Thank You!